Internal Working of HashMap

What is HashMap?

HashMap is a data structure in Java that stores key-value pairs. It implements the Map interface and uses hashing to store and retrieve data efficiently. HashMap allows one null key and multiple null values, and it's not synchronized (not thread-safe).

```java
import java.util.HashMap;

HashMap<String, Integer> map = new HashMap<>();
map.put("Apple", 10);
map.put("Banana", 20);
map.put("Orange", 15);
```

Hash Function and Hashing

What is Hashing?

Hashing is the process of converting a key into an array index using a hash function. The hash function takes the key and produces an integer value (hash code) that determines where the key-value pair will be stored in the internal array.

```java
public class HashExample {
    public static void main(String[] args) {
        String key = "Hello";
        int hashCode = key.hashCode();
        System.out.println("Hash code of 'Hello': " + hashCode);

        // The actual index is calculated as:
        // index = hashCode % array_length
    }
}
```

Hash Function in HashMap

HashMap uses the `hashCode()` method of the key object to generate hash codes. For better distribution, HashMap applies additional processing to the hash code:

```java
// Simplified version of HashMap's hash function
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

Internal Structure - Array of Buckets

Bucket Array

Internally, HashMap maintains an array of buckets (also called bins). Each bucket can store multiple key-value pairs when hash collisions occur.

```java
// Conceptual representation of HashMap's internal structure
class HashMapInternal {
    Node<K,V>[] table;  // Array of buckets
    int size;           // Number of key-value pairs
    int threshold;      // Resize threshold
    float loadFactor;   // Load factor (default 0.75)
}
```

Node Structure

Each entry in HashMap is stored as a Node that contains:

- Key
- Value
- Hash code
- Reference to next node (for collision handling)

```java
static class Node<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next;

    Node(int hash, K key, V value, Node<K,V> next) {
        this.hash = hash;
        this.key = key;
        this.value = value;
        this.next = next;
    }
}
```

Hash Collisions and Resolution

What are Hash Collisions?

Hash collisions occur when two different keys produce the same hash code or map to the same bucket index. HashMap handles collisions using chaining and tree structures.

```java
public class CollisionExample {
    public static void main(String[] args) {
        // These strings have the same hash code
        String str1 = "Aa";
        String str2 = "BB";

        System.out.println(str1.hashCode()); // 2112
        System.out.println(str2.hashCode()); // 2112

        HashMap<String, Integer> map = new HashMap<>();
        map.put(str1, 1);
        map.put(str2, 2); // Collision will occur
    }
}
```

Collision Resolution Methods

1. Chaining (Linked List)

When collisions occur, HashMap initially uses a linked list to store multiple entries in the same bucket.

```java
// Bucket with collision - linked list structure
// Bucket[5]: Node1 -> Node2 -> Node3 -> null
//           (key1)   (key2)   (key3)
```

2. Tree Structure (Red-Black Tree)

When a bucket has more than 8 entries, HashMap converts the linked list to a balanced red-black tree for better performance.

```java
// When linked list length > TREEIFY_THRESHOLD (8)
// Linked list converts to Red-Black Tree
static final int TREEIFY_THRESHOLD = 8;
static final int UNTREEIFY_THRESHOLD = 6;
```

Load Factor and Resizing

Load Factor

Load factor determines when HashMap should resize its internal array. Default load factor is 0.75, meaning when 75% of buckets are filled, HashMap will resize.

```java
HashMap<String, Integer> map = new HashMap<>(16, 0.75f);
// Initial capacity: 16
// Load factor: 0.75
// Resize threshold: 16 * 0.75 = 12
```

Resizing Process

When the number of entries exceeds the threshold, HashMap doubles its capacity and rehashes all existing entries.

```java
public class ResizingExample {
    public static void main(String[] args) {
        HashMap<Integer, String> map = new HashMap<>(4);

        // Initial capacity: 4, threshold: 3
        map.put(1, "One");   // Size: 1
        map.put(2, "Two");   // Size: 2
        map.put(3, "Three"); // Size: 3
        map.put(4, "Four");  // Size: 4 - Triggers resize!

        // New capacity: 8, threshold: 6
    }
}
```

Rehashing

During resize, all existing entries are rehashed and redistributed across the new larger array.

```java
// Rehashing process
void resize() {
    Node<K,V>[] oldTab = table;
    int newCap = oldTab.length * 2;  // Double the capacity
    Node<K,V>[] newTab = new Node[newCap];

    // Rehash all existing entries
    for (Node<K,V> e : oldTab) {
        while (e != null) {
            int newIndex = e.hash & (newCap - 1);
            // Place entry in new position
        }
    }
}
```

Put Operation Step-by-Step

Algorithm for put() method

```java
public V put(K key, V value) {
    // Step 1: Calculate hash
    int hash = hash(key);

    // Step 2: Find bucket index
    int index = hash & (table.length - 1);

    // Step 3: Handle collision or insert
    Node<K,V> node = table[index];

    if (node == null) {
        // No collision - create new node
        table[index] = new Node(hash, key, value, null);
    } else {
        // Collision - traverse chain or tree
        // Update existing key or add new node
    }

    // Step 4: Check if resize needed
    if (++size > threshold) {
        resize();
    }

    return oldValue;
}
```

Example Put Operation

```java
public class PutExample {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();

        // put("Java", 100)
        // 1. hash("Java") = 2301506
        // 2. index = 2301506 & 15 = 10
        // 3. table[10] = new Node("Java", 100)
        map.put("Java", 100);

        // put("Python", 200)
        // 1. hash("Python") = -1774097
        // 2. index = -1774097 & 15 = 3
        // 3. table[3] = new Node("Python", 200)
        map.put("Python", 200);
    }
}
```

Get Operation Step-by-Step

Algorithm for get() method

```java
public V get(Object key) {
    // Step 1: Calculate hash
    int hash = hash(key);

    // Step 2: Find bucket index
    int index = hash & (table.length - 1);

    // Step 3: Search in bucket
    Node<K,V> node = table[index];

    while (node != null) {
        if (node.hash == hash &&
            (node.key == key || key.equals(node.key))) {
            return node.value;  // Found!
        }
        node = node.next;  // Move to next node in chain
    }

    return null;  // Not found
}
```

Example Get Operation

```java
public class GetExample {
    public static void main(String[] args) {
        HashMap<String, Integer> map = new HashMap<>();
        map.put("Java", 100);
        map.put("Python", 200);

        // get("Java")
        // 1. hash("Java") = 2301506
        // 2. index = 2301506 & 15 = 10
        // 3. Search table[10] chain
        // 4. Found node with key="Java", return 100
        Integer value = map.get("Java");
        System.out.println(value); // Output: 100
    }
}
```

Performance Analysis

Time Complexity

Operation	Average Case	Worst Case
put()	O(1)	O(n)
get()	O(1)	O(n)
remove()	O(1)	O(n)

Best Case Performance

When hash function distributes keys evenly and no collisions occur:

```java
// Perfect distribution - O(1) operations
HashMap<Integer, String> map = new HashMap<>();
map.put(1, "One");    // O(1)
map.put(2, "Two");    // O(1)
map.put(3, "Three");  // O(1)

String value = map.get(2);  // O(1)
```

Worst Case Performance

When all keys hash to the same bucket (before Java 8 tree optimization):

```java
// All keys collide - O(n) operations
HashMap<BadHashKey, String> map = new HashMap<>();
// If BadHashKey always returns same hash code
// All entries go to same bucket -> O(n) search
```

Tree Optimization (Java 8+)

With tree structure for large collision chains:

```java
// When bucket has > 8 entries, converts to tree
// Search becomes O(log n) instead of O(n)
static final int TREEIFY_THRESHOLD = 8;

// Tree operations: O(log n) in worst case
```

Complete Working Example

java

```java
import java.util.HashMap;

public class HashMapDemo {
    public static void main(String[] args) {
        // Create HashMap with initial capacity 4
        HashMap<String, Integer> studentGrades = new HashMap<>(4);

        System.out.println("=== Adding Students ===");

        // Add student grades
        studentGrades.put("Alice", 85);    // Hash + Store
        studentGrades.put("Bob", 92);      // Hash + Store
        studentGrades.put("Charlie", 78);  // Hash + Store
        studentGrades.put("Diana", 96);    // Hash + Store (triggers resize)
        studentGrades.put("Eve", 88);      // Hash + Store in resized map

        System.out.println("Total students: " + studentGrades.size());

        System.out.println("\n=== Retrieving Grades ===");

        // Retrieve grades
        System.out.println("Alice's grade: " + studentGrades.get("Alice"));
        System.out.println("Bob's grade: " + studentGrades.get("Bob"));
        System.out.println("Frank's grade: " + studentGrades.get("Frank")); // null

        System.out.println("\n=== Updating Grades ===");

        // Update existing grade
        Integer oldGrade = studentGrades.put("Alice", 90);
        System.out.println("Alice's old grade: " + oldGrade);
        System.out.println("Alice's new grade: " + studentGrades.get("Alice"));

        System.out.println("\n=== Checking Existence ===");

        // Check if key exists
        if (studentGrades.containsKey("Charlie")) {
            System.out.println("Charlie is in the class");
        }

        // Check if value exists
        if (studentGrades.containsValue(96)) {
            System.out.println("Someone got 96 points");
        }
```

```
        System.out.println("\n=== All Students and Grades ===");

        // Iterate through all entries
        for (String student : studentGrades.keySet()) {
            System.out.println(student + ": " + studentGrades.get(student));
        }
    }
}
```

Quiz Questions

1. What is the default load factor of HashMap in Java? a) 0.5 b) 0.75 c) 1.0 d) 0.25

Answer: b) 0.75

2. When does HashMap convert a linked list to a tree structure? a) When the bucket has more than 6 entries b) When the bucket has more than 8 entries c) When the bucket has more than 10 entries d) When the load factor exceeds 0.75

Answer: b) When the bucket has more than 8 entries

3. What happens when two keys have the same hash code in HashMap? a) The second key overwrites the first key b) An exception is thrown c) Hash collision occurs and chaining is used d) The HashMap automatically resizes

Answer: c) Hash collision occurs and chaining is used

4. What is the average time complexity for get() operation in HashMap? a) O(1) b) O(log n) c) O(n) d) O(n log n)

Answer: a) O(1)

5. When does HashMap resize its internal array? a) When all buckets are filled b) When the number of entries exceeds the threshold (capacity × load factor) c) When a collision occurs d) When the load factor becomes 1.0

Answer: b) When the number of entries exceeds the threshold (capacity × load factor)

Practice Coding Questions

1. Custom Hash Function Write a Java program that demonstrates how different hash functions affect the distribution of keys in a HashMap. Create a custom class with a poor hash function that always

returns the same value, and compare its performance with the default hash function.

2. Collision Analysis Create a program that finds pairs of strings that produce hash collisions (same hash code). Use these strings as keys in a HashMap and demonstrate how HashMap handles the collisions internally.

3. Load Factor Experiment Write a program that creates multiple HashMaps with different load factors (0.25, 0.5, 0.75, 1.0) and measures the performance difference when inserting and retrieving a large number of elements.

4. Rehashing Simulation Implement a simplified version of HashMap's resize operation. Create a program that shows the step-by-step process of rehashing entries when the HashMap needs to expand its capacity.

5. Performance Comparison Create a performance comparison program that tests HashMap operations against other data structures (ArrayList, LinkedList, TreeMap) for different scenarios: frequent insertions, frequent lookups, and frequent deletions. Measure and display the time taken for each operation.